

AI Hardware & System Performance Analysis: Complete Guide

For SemiAnalysis Research Analyst Role

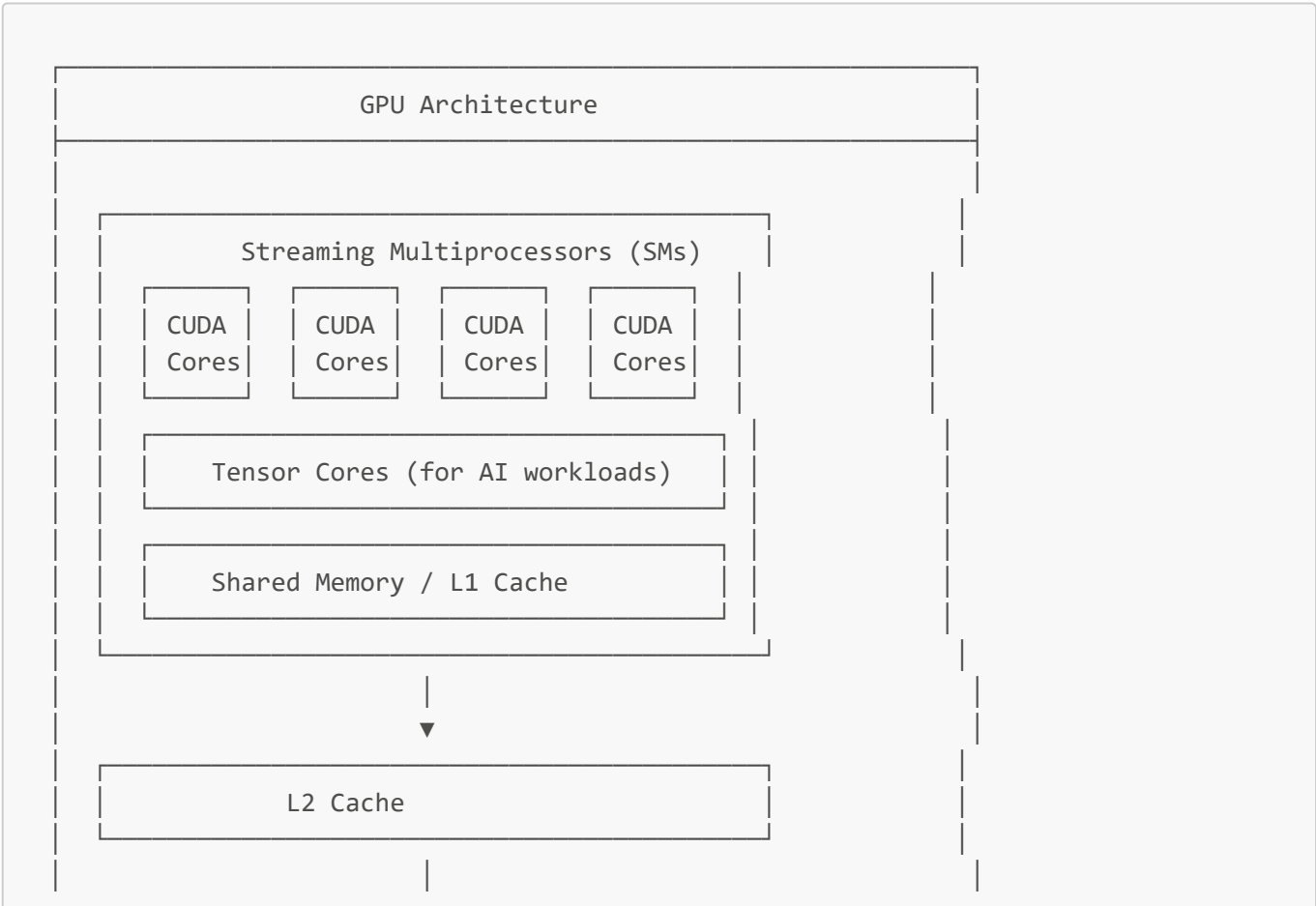
Table of Contents

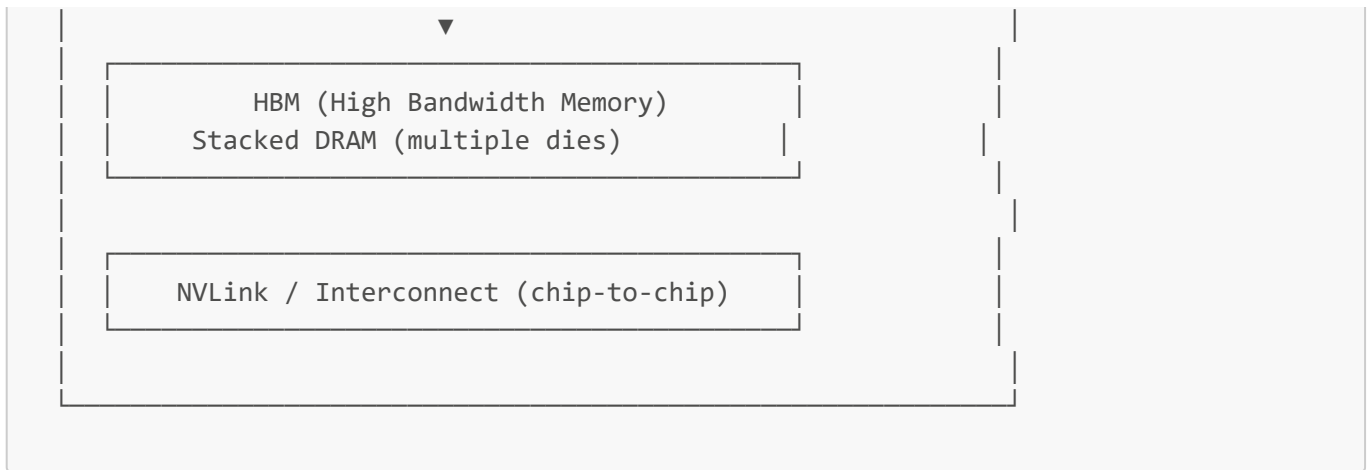
- 1. AI Hardware Architecture Fundamentals
- 2. Performance Analysis & Modeling
- 3. LLM Inference & Training Deep Dive
- 4. Semiconductor Manufacturing & Economics
- 5. Datacenter Infrastructure & TCO
- 6. Memory Systems (HBM, SRAM, Tiering)
- 7. Networking & Interconnects
- 8. Power, Cooling & Efficiency
- 9. Real-World Case Studies
- 10. Interview Questions (50)

1. AI Hardware Architecture Fundamentals

GPU Architecture (NVIDIA, AMD)

Key Components:





NVIDIA H100 Specs Example:

Compute:

- 80 SMs (Streaming Multiprocessors)
- 16,896 CUDA cores
- 528 Tensor Cores (4th gen)
- FP16 Tensor: 1,979 TFLOPS
- FP8 Tensor: 3,958 TFLOPS
- FP64: 67 TFLOPS

Memory:

- 80GB HBM3
- 3.35 TB/s bandwidth
- 50 MB L2 cache

Interconnect:

- NVLink: 900 GB/s (18 links × 50 GB/s each)
- PCIe Gen5: 128 GB/s

Power:

- TDP: 700W

Manufacturing:

- TSMC 4N (Custom 5nm)
- Die size: ~814 mm²
- Transistors: 80 billion
- CoWoS packaging (2.5D)

Tensor Core Architecture

What are Tensor Cores?

Specialized hardware for matrix multiplication:

Traditional CUDA Core:

- Scalar operations ($a \times b$)
- One operation per clock

- Tensor Core:
- Matrix operations (4x4 matrix multiply)
 - 64 operations per clock (16x speedup)

Example:

A =

a	b	c	d
e	f	g	h
i	j	k	l
m	n	o	p

B =

w	x	y	z
w	x	y	z
w	x	y	z
w	x	y	z

- C = A x B (4x4 x 4x4 matrix multiply)
- 1 Tensor Core operation (1 clock cycle)
 - 64 FP16 multiply-accumulate operations
 - Throughput: 64 ops/cycle x frequency

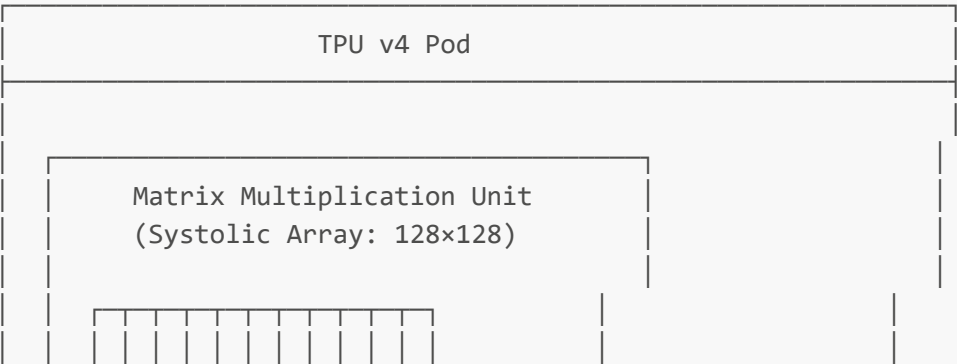
Generations:

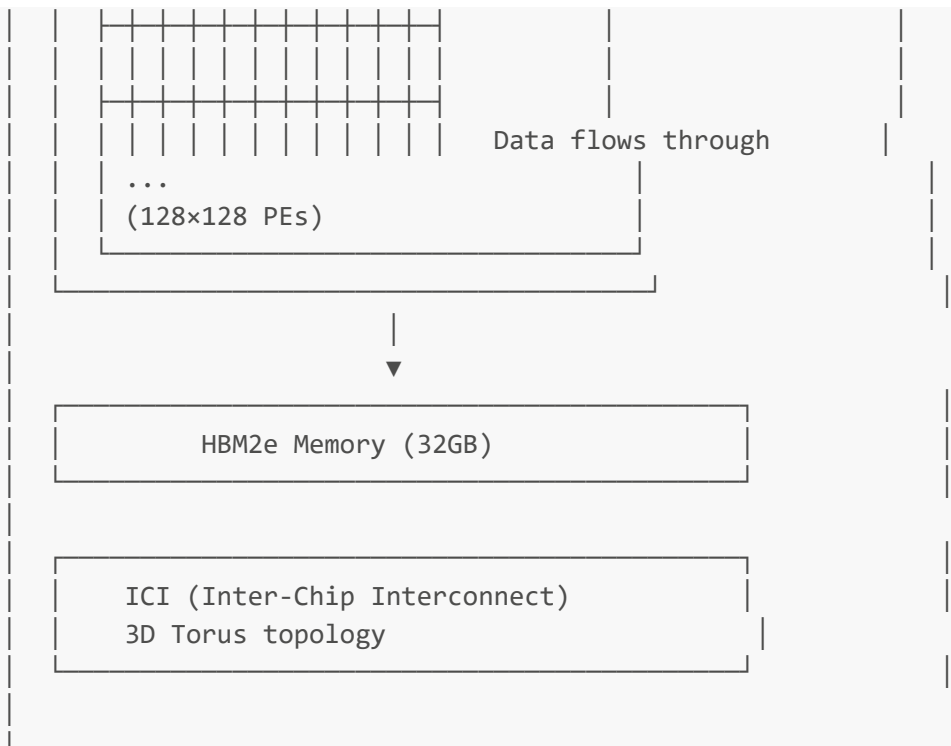
Generation	GPU	Precision	TFLOPS (FP16)
1st Gen	V100	FP16, FP32	125 TFLOPS
2nd Gen	A100	+ BF16, TF32	312 TFLOPS
3rd Gen	A100 (80G)	+ FP64	312 TFLOPS
4th Gen	H100	+ FP8	1,979 TFLOPS
		FP8	3,958 TFLOPS

- Key Innovation: FP8 (H100+)
- 2x throughput vs FP16
 - Transformer-optimized
 - Minimal accuracy loss with proper scaling

TPU Architecture (Google)

TPU v4 Architecture:





Specs:

- BF16/FP32: 275 TFLOPS (per chip)
- INT8: 1,100 TOPS
- Memory: 32GB HBM2e, 1.2 TB/s
- Interconnect: 4x custom chip-to-chip links
- Power: ~300W per chip
- Pod: 4,096 chips (1.1 exaFLOPS)

Systolic Array Concept:

Data flows through Processing Elements (PEs):

	a_1	a_2	a_3	a_4	
	↓	↓	↓	↓	
b_1	→ PE	→ PE	→ PE	→ PE	→ result ₁
b_2	→ PE	→ PE	→ PE	→ PE	→ result ₂
b_3	→ PE	→ PE	→ PE	→ PE	→ result ₃
b_4	→ PE	→ PE	→ PE	→ PE	→ result ₄

Each PE:

- Receives weight from above
- Receives activation from left
- Computes: $\text{accumulate} += \text{weight} \times \text{activation}$
- Passes data to next PE

Advantage:

- High throughput (all PEs active)
- Low memory bandwidth (data reuse)
- Deterministic performance

Custom ASICs (AWS Trainium, Cerebras, Groq)

AWS Trainium

Optimized for training:

- NeuronCores: Custom matrix engines
- FP32, BF16, FP8, cFP8 (configurable FP8)
- 512GB HBM2e (Trn1.32xlarge = 16 chips)
- NeuronLink: 768 Gbps per chip
- EFA (Elastic Fabric Adapter) networking

Key Difference:

- Software-defined precision
- Compiler-optimized for PyTorch/JAX
- Cost-optimized vs NVIDIA

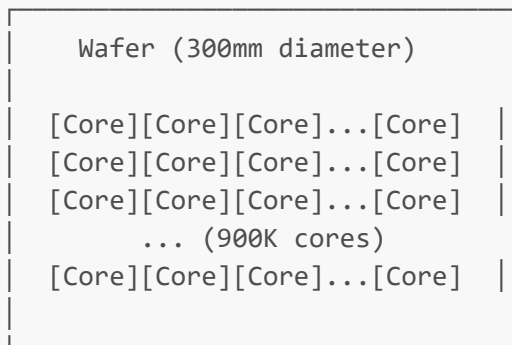
Cerebras WSE-3 (Wafer Scale Engine)

Entire wafer = single chip!

Specs:

- 900,000 AI cores
- 44 GB on-chip SRAM
- 21 PB/s memory bandwidth
- Die size: 46,225 mm² (whole wafer!)
- 4 trillion transistors

Architecture:



Advantage:

- Zero off-chip memory access
- Massive on-chip bandwidth
- Perfect for large models

Challenge:

- Yield (defects across wafer)
- Cooling (entire wafer gets hot)
- Limited to single wafer

Groq LPU (Language Processing Unit)

Software-defined hardware:

Key Innovation:

- Deterministic execution
- No cache (all SRAM)
- Compiler determines data flow
- Low batch size, high throughput

Architecture:

- 230 MB on-chip SRAM
- Functional units: 320
- Clock: 900 MHz
- No memory hierarchy (flat SRAM)

Use Case:

- Real-time inference
- Low latency > high throughput
- Single-token generation

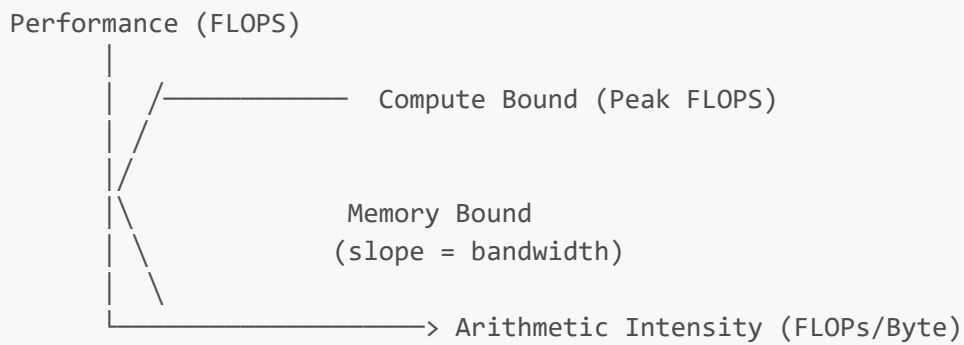
Architecture Comparison

Architecture	Strength	Weakness	Best For	Cost/Perf
NVIDIA GPU	Flexible Software Ecosystem	Power Cost	Training Inference Research	Baseline
Google TPU	Training Perf Cost	Lock-in Software Rigid	Large-scale Training Google	Lower
AWS Trainium	Cost	Immature Software	Training AWS users	Lowest
Cerebras WSE	Bandwidth Latency	Yield Scaling	Giant Models	Very High
Groq LPU	Latency Determin	Batch Throughput	Real-time Inference	TBD

2. Performance Analysis & Modeling

Roofline Model

Concept: Peak performance limited by compute OR memory bandwidth.



Arithmetic Intensity = Operations / Memory Traffic

Example:

- Matrix Multiply (large): High AI → Compute bound
- Element-wise ops: Low AI → Memory bound

Calculation:

```
def roofline_performance(
    arithmetic_intensity, # FLOPs per byte
    peak_flops,          # Hardware peak FLOPS
    memory_bandwidth     # Bytes/sec
):
    """
    Roofline model: Performance limited by compute or memory
    """
    # Memory-bound performance
    memory_limited = arithmetic_intensity * memory_bandwidth

    # Compute-bound performance
    compute_limited = peak_flops

    # Actual performance is minimum
    actual_performance = min(memory_limited, compute_limited)

    return actual_performance

# Example: NVIDIA H100
h100_peak_flops = 1979e12 # 1,979 TFLOPS FP16
h100_bandwidth = 3.35e12 # 3.35 TB/s

# Matrix multiply: AI = 100 FLOPs/Byte
matmul_perf = roofline_performance(100, h100_peak_flops, h100_bandwidth)
print(f"MatMul: {matmul_perf/1e12:.1f} TFLOPS") # ~335 TFLOPS (compute-bound)

# Element-wise: AI = 0.5 FLOPs/Byte
elementwise_perf = roofline_performance(0.5, h100_peak_flops, h100_bandwidth)
```

```
print(f"Element-wise: {elementwise_perf/1e12:.1f} TFLOPS") # ~1.7 TFLOPS (memory-bound)
```

Bottleneck Analysis:

Ridge Point = Peak FLOPS / Memory Bandwidth

H100:

Ridge Point = 1,979 TFLOPS / 3.35 TB/s = 591 FLOPs/Byte

If AI < 591: Memory-bound

If AI > 591: Compute-bound

Most transformer operations:

- Attention QK^T: AI ≈ 50-100 (memory-bound)
- Attention softmax: AI ≈ 1 (very memory-bound!)
- FFN matmul: AI ≈ 100-500 (depends on batch size)

Model FLOPs Utilization (MFU)

Definition: Fraction of peak hardware FLOPS actually achieved.

MFU = Achieved FLOPS / Peak FLOPS

Example:

Hardware: H100 @ 1,979 TFLOPS peak

Achieved: 600 TFLOPS during training

MFU = 600 / 1,979 = 30%

Industry benchmarks:

- Good training: 40-60% MFU
- Excellent training: 60-70% MFU
- Record (optimized): 70%+ MFU

Why not 100%?

Losses:

1. Memory-bound operations (30-50%)
 - Attention, softmax, LayerNorm
2. Communication overhead (10-20%)
 - All-reduce, all-gather for distributed training
3. CPU overhead (5-10%)
 - Data loading, preprocessing
4. Framework overhead (5-10%)

- PyTorch, gradient computation

5. Pipeline bubbles (distributed training)

- Idle time waiting for other GPUs

Measurement:

```
def calculate_mfu(
    model_params,      # Number of parameters
    tokens_per_sec,    # Training throughput
    sequence_length,   # Token sequence length
    peak_flops         # Hardware peak FLOPS
):
    """
    Calculate Model FLOPs Utilization

    FLOPs per token ≈ 6 × params (for transformers)
    """
    # FLOPs for forward + backward pass
    flops_per_token = 6 * model_params * sequence_length

    # Total FLOPs/sec achieved
    achieved_flops = flops_per_token * tokens_per_sec

    # MFU
    mfu = achieved_flops / peak_flops

    return mfu

# Example: Training GPT-3 (175B) on H100
gpt3_params = 175e9
tokens_per_sec = 5000 # 5K tokens/sec per GPU
seq_len = 2048
h100_peak = 1979e12

mfu = calculate_mfu(gpt3_params, tokens_per_sec, seq_len, h100_peak)
print(f"MFU: {mfu*100:.1f}%")
```

Tokens per Second per Watt

Energy Efficiency Metric:

Efficiency = Tokens/sec / Power (W)

Example:

H100:

- Inference: 10,000 tokens/sec
- Power: 700W
- Efficiency: 14.3 tokens/sec/watt

A100:

- Inference: 6,000 tokens/sec
- Power: 400W
- Efficiency: 15 tokens/sec/watt

Winner: A100 (more efficient)
But H100 has higher absolute throughput!

TCO Impact:

Datacenter costs over 3 years:

Metric	H100 (8x)	A100 (8x)
Hardware Cost	\$240,000	\$80,000
Power (3yr)	\$50,000	\$30,000
Cooling (3yr)	\$15,000	\$9,000
Throughput	80K tok/s	48K tok/s
Total Cost	\$305,000	\$119,000
Cost per token	\$0.00000382	\$0.00000248

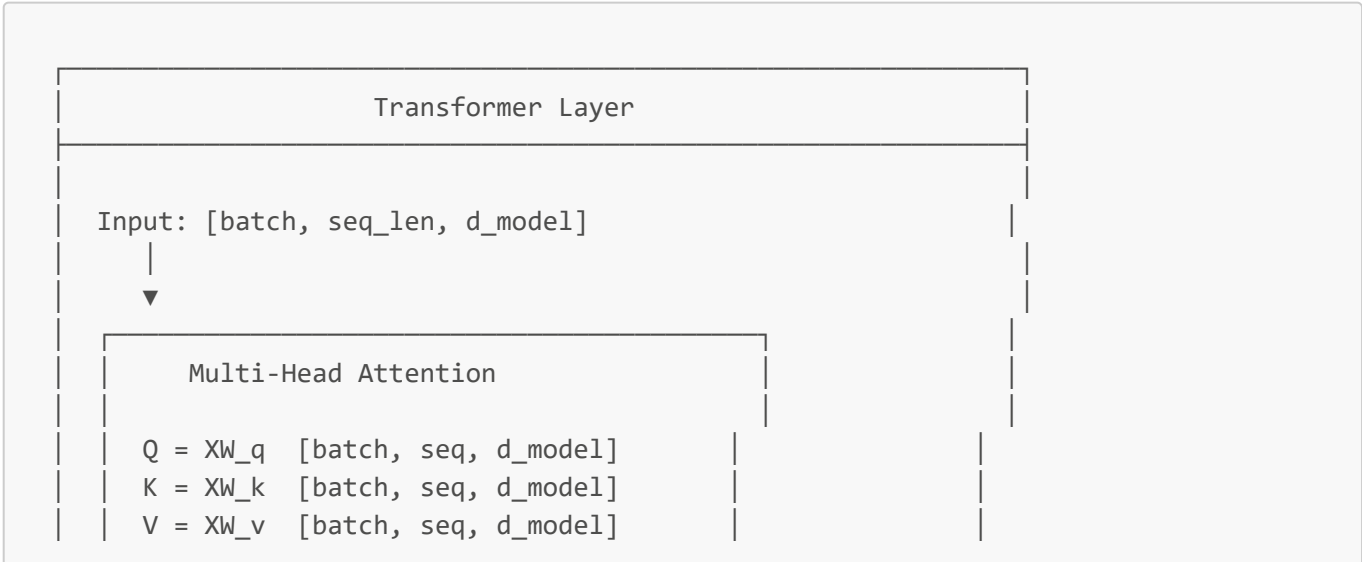
A100 is cheaper per token!
But H100 delivers 67% more throughput.

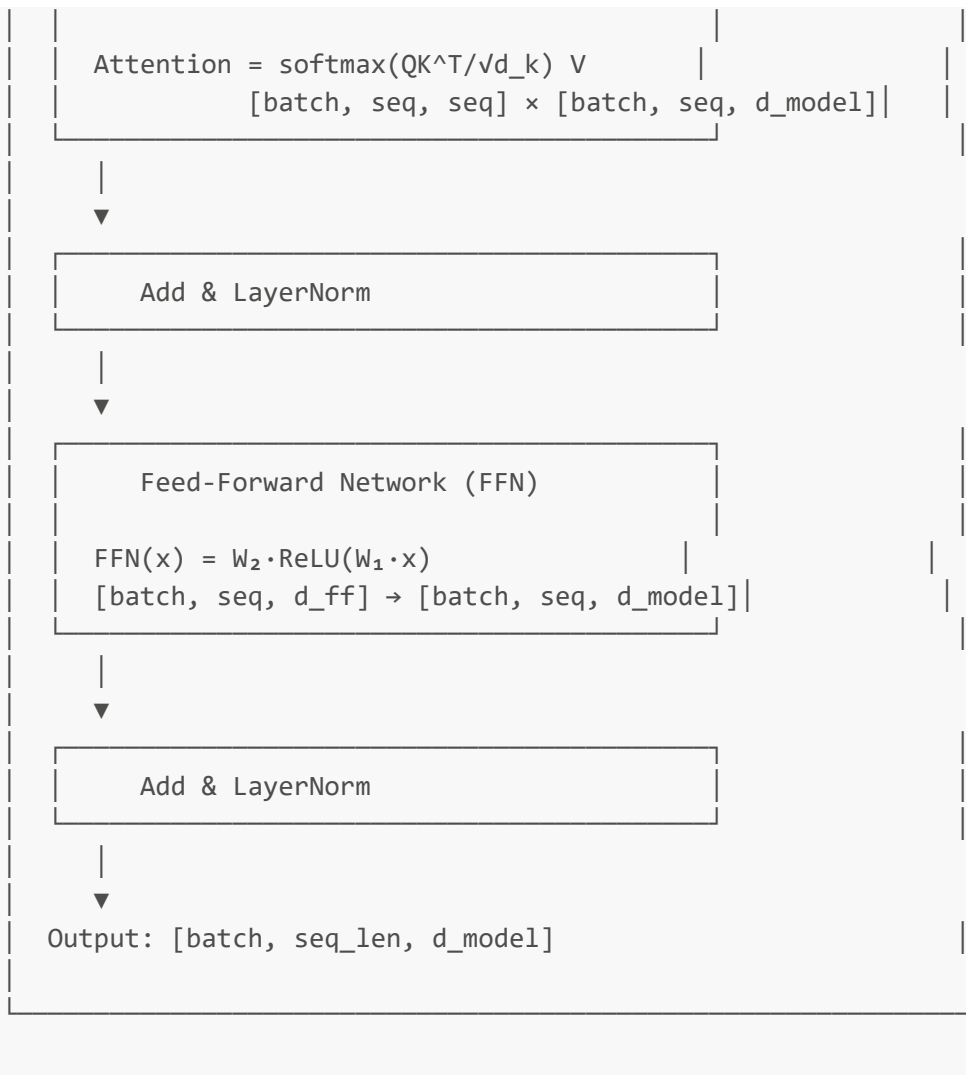
Decision depends on:

- Need for throughput vs efficiency
- Amortization period
- Workload characteristics

3. LLM Inference & Training Deep Dive

Transformer Architecture Review





Prefill vs Decode Phase

Prefill (Context Processing):

Input: Entire prompt (e.g., 2048 tokens)
Output: KV cache for all tokens

Characteristics:

- Parallel computation (all tokens at once)
- Compute-bound (large matrix multiplications)
- High throughput
- Memory bandwidth: Moderate
- Latency: ~100ms for 2K tokens

FLOPs per token: $2 \times \text{params} \times \text{seq_len}$

Example (LLaMA 70B, 2K context):

$$\text{FLOPs} = 2 \times 70\text{B} \times 2048 = 287 \text{ TFLOPs}$$

On H100 (1979 TFLOPS peak):

$$\text{Time} = 287 / 1979 \approx 0.145 \text{ sec} = 145\text{ms}$$

Decode (Token Generation):

Input: 1 token + existing KV cache
 Output: Next token + updated KV cache

Characteristics:

- Sequential (one token at a time)
- Memory-bound (loading KV cache)
- Low throughput per request
- High memory bandwidth needed
- Latency: ~10-50ms per token

FLOPs per token: $2 \times \text{params}$

Example (LLaMA 70B):

$$\text{FLOPs} = 2 \times 70\text{B} = 140 \text{ GFLOPs}$$

But bottleneck is KV cache bandwidth!

KV Cache Analysis

Memory Requirements:

```
def kv_cache_size(
    batch_size,
    sequence_length,
    num_layers,
    num_heads,
    head_dim,
    bytes_per_element=2 # FP16
):
    """
    Calculate KV cache memory

    Each layer stores:
    - K: [batch, num_heads, seq_len, head_dim]
    - V: [batch, num_heads, seq_len, head_dim]
    """
    # 2x for K and V
    size_bytes = (
        2 * batch_size * sequence_length *
        num_layers * num_heads * head_dim *
        bytes_per_element
    )

    return size_bytes

# Example: LLaMA 70B
llama_70b_cache = kv_cache_size(
    batch_size=1,
    sequence_length=4096, # 4K context
    num_layers=80,
    num_heads=64,
```

```

    head_dim=128,
    bytes_per_element=2 # FP16
)

print(f"KV cache: {llama_70b_cache / 1e9:.2f} GB")
# Output: 5.24 GB per request

# For batch_size=32:
print(f"Batch 32: {llama_70b_cache * 32 / 1e9:.2f} GB")
# Output: 167.77 GB (exceeds H100 80GB!)

```

KV Cache Bandwidth:

During decode, must load entire KV cache each step:

Bandwidth needed = KV_cache_size / time_per_token

Example:

- KV cache: 5 GB
- Target: 100 tokens/sec (10ms per token)
- Bandwidth: 5 GB / 0.01s = 500 GB/s

H100 bandwidth: 3,350 GB/s ✓ (sufficient)

A100 bandwidth: 1,935 GB/s ✓ (sufficient)

Batching Strategies

Static Batching:

All requests same length:

Batch 1: [req1: 100 tokens, req2: 100 tokens, req3: 100 tokens]

Batch 2: [req4: 100 tokens, req5: 100 tokens, req6: 100 tokens]

Problem:

- Padding waste if requests different lengths
- Wait for slowest request in batch

Continuous Batching (vLLM):

Add/remove requests dynamically:

Time T0: [req1____req2____req3____]
 Generate Generate Generate

Time T1: [req1____req2____req3____req4__]
 req1 done, req4 added immediately

Advantages:

- No padding waste
- Higher GPU utilization
- Lower latency

Implementation:

```
# Simplified continuous batching
class ContinuousBatcher:
    def __init__(self, max_batch_size, max_seq_len):
        self.max_batch_size = max_batch_size
        self.max_seq_len = max_seq_len
        self.active_requests = []

    def add_request(self, request):
        """Add new request to batch"""
        if len(self.active_requests) < self.max_batch_size:
            self.active_requests.append(request)
            return True
        return False

    def generate_step(self):
        """Generate one token for all active requests"""
        if not self.active_requests:
            return

        # Prepare batch
        batch = self.prepare_batch(self.active_requests)

        # Generate tokens
        output_tokens = model.forward(batch)

        # Update requests
        completed = []
        for i, request in enumerate(self.active_requests):
            request.add_token(output_tokens[i])

            if request.is_complete():
                completed.append(i)

        # Remove completed requests
        for i in reversed(completed):
            self.active_requests.pop(i)
```

PagedAttention (vLLM)

Problem: KV cache memory fragmentation

Traditional allocation:

Request 1: [_____] 2KB allocated, 1KB used (50% waste)

Request 2: [_____] 3KB allocated, 0.5KB used (83% waste!)

Solution: Paged memory (like OS virtual memory)

Page1	Page2	Page3	Page4	Fixed-size blocks

Request 1: Page1 → Page2 (fill as needed)

Request 2: Page3 → Page4 (fill as needed)

- Benefits:
- Near-zero waste
 - 2-4× more concurrent requests
 - Dynamic allocation

4. Semiconductor Manufacturing & Economics

Process Nodes & Technology

Node Evolution:

Node	Year	Foundry	Gate Pitch	Transistor Density
7nm	2018	TSMC	54nm	96 MTr/mm ²
5nm	2020	TSMC	48nm	171 MTr/mm ²
4nm	2022	TSMC	48nm	171 MTr/mm ²
3nm	2022	TSMC	48nm	292 MTr/mm ²
4N	2022	TSMC (NVIDIA)	Custom	~180 MTr/mm ²

- Key Insight:
- "4nm" is marketing (same as 5nm with optimizations)
 - True scaling: 7nm → 5nm → 3nm
 - NVIDIA 4N: Custom TSMC 5nm optimized for H100

Process Node Comparison:

Metric	7nm	5nm	3nm	2nm(future)
Density	100%	171%	292%	~480%
Power (same performance)	100%	70%	60%	~45%

Cost/wafer	~\$10K	~\$16K	~\$20K	~\$25K+
Yield	~90%	~85%	~80%	TBD
Design cost	\$300M	\$500M	\$700M+	\$1B+

Trade-offs:

Advanced nodes:

- ✓ Higher density (more transistors)
- ✓ Better power efficiency
- ✗ Much higher cost per wafer
- ✗ Lower yields
- ✗ Higher design costs

Die Size & Reticle Limits

Reticle Limit:

Photolithography reticle size: 858 mm² (33mm × 26mm)

Single die can't exceed reticle limit!

Examples:

Reticle (858 mm²)

NVIDIA H100

814 mm²

(Fits in one shot!)

If larger than reticle:

- Must use advanced packaging (CoWoS, chiplets)
- Or multiple smaller dies

Die Size Economics:

```
def calculate_dies_per_wafer(die_size_mm2, wafer_diameter_mm=300):  
    """  
    Approximate dies per wafer  
  
    Uses: Wafer area / Die area (simplified)  
    Real formula accounts for edge loss  
    """  
    wafer_radius = wafer_diameter_mm / 2  
    wafer_area = 3.14159 * wafer_radius ** 2  
  
    # Edge loss (5-10% of dies unusable)  
    usable_area = wafer_area * 0.93
```

```

    dies_per_wafer = int(usable_area / die_size_mm2)

    return dies_per_wafer

# Examples:
h100_dies = calculate_dies_per_wafer(814) # H100: 814 mm²
print(f"H100 dies per wafer: {h100_dies}") # ~80 dies

a100_dies = calculate_dies_per_wafer(826) # A100: 826 mm²
print(f"A100 dies per wafer: {a100_dies}") # ~78 dies

# Smaller die example:
small_dies = calculate_dies_per_wafer(200) # 200 mm²
print(f"Small dies per wafer: {small_dies}") # ~327 dies

```

Yield Calculation

Defect Density:

$$\text{Yield} = (1 + (\text{Defects} \times \text{Die_Area}) / \alpha)^{-\alpha}$$

Where:

- Defects: Defects per cm² (D0)
- Die_Area: Die size in cm²
- α : Process complexity factor (~4-8)

Intuition:

- Larger die → lower yield
- More defects → lower yield
- Advanced process → more defects

Calculation:

```

def calculate_yield(die_area_mm2, defect_density=0.09, alpha=6):
    """
    Calculate die yield using Murphy's model

    Args:
        die_area_mm2: Die area in mm²
        defect_density: Defects per cm² (typical: 0.05-0.15)
        alpha: Complexity factor (4-8, higher = more complex)
    """
    # Convert mm² to cm²
    die_area_cm2 = die_area_mm2 / 100

    # Murphy's yield model
    yield_fraction = (1 + (defect_density * die_area_cm2) / alpha) ** (-alpha)

```

```

    return yield_fraction

# Examples:
h100_yield = calculate_yield(814, defect_density=0.09, alpha=6)
print(f"H100 yield (814 mm²): {h100_yield:.1%}") # ~70-80%

small_die_yield = calculate_yield(200, defect_density=0.09, alpha=6)
print(f"Small die yield (200 mm²): {small_die_yield:.1%}") # ~92%

# Sensitivity analysis:
print("\nYield vs Die Size:")
for size in [100, 200, 400, 600, 814, 1000]:
    y = calculate_yield(size, defect_density=0.09)
    print(f" {size} mm²: {y:.1%}")

# Output:
# 100 mm²: 95.7%
# 200 mm²: 91.8%
# 400 mm²: 84.6%
# 600 mm²: 78.5%
# 814 mm²: 71.9%
# 1000 mm²: 67.1%

```

Yield Impact on Cost:

Example: H100

Wafer cost: \$16,000

Dies per wafer: 80

Yield: 75%

Good dies per wafer: $80 \times 0.75 = 60$

Die cost: $\$16,000 / 60 = \267 per die

If yield drops to 60%:

Good dies: $80 \times 0.60 = 48$

Die cost: $\$16,000 / 48 = \333 per die (+25% cost!)

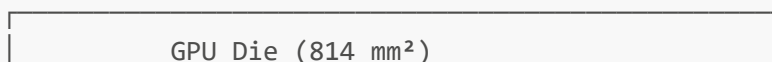
Small improvement in yield = big cost savings!

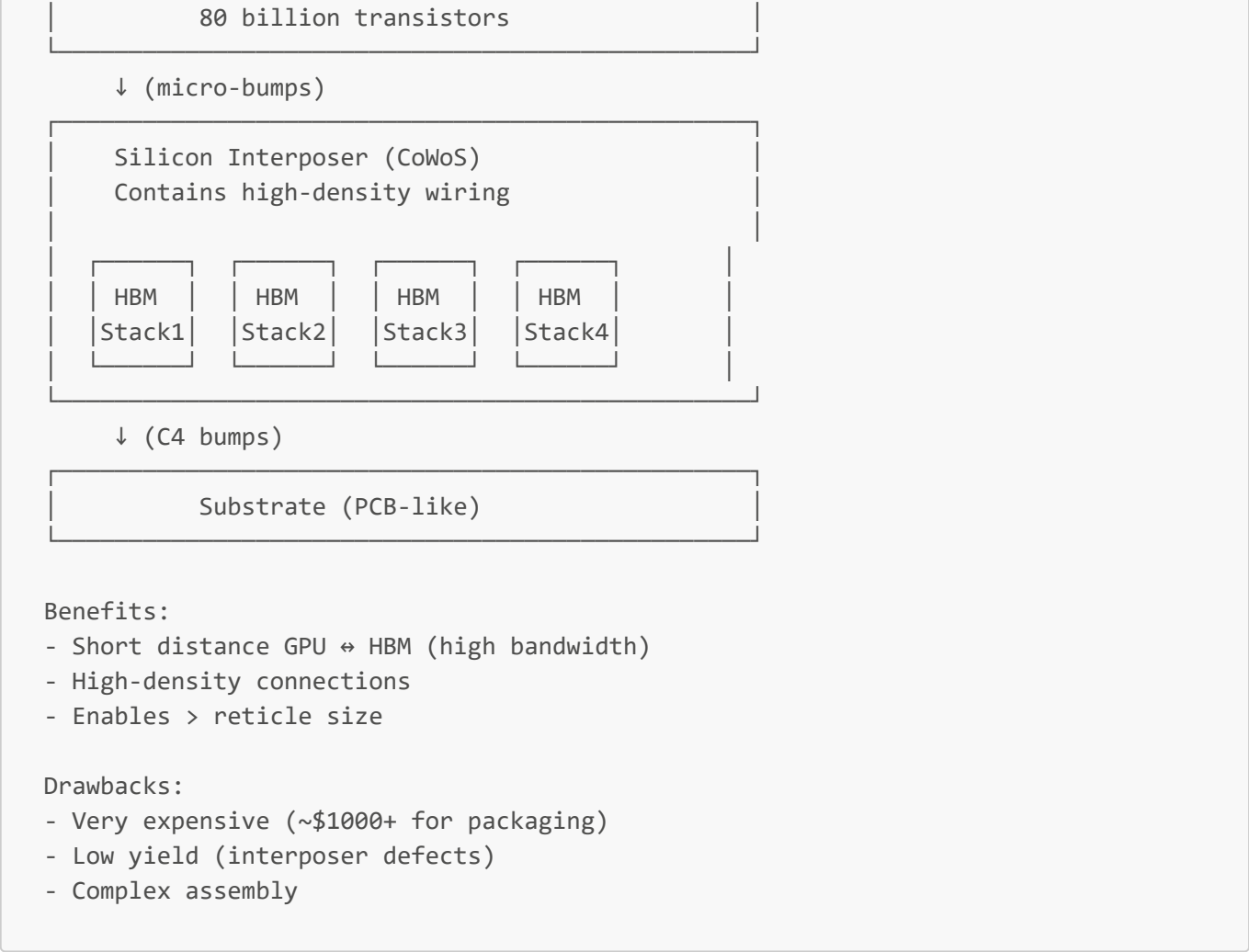
Advanced Packaging

CoWoS (Chip-on-Wafer-on-Substrate)

NVIDIA H100 uses CoWoS:

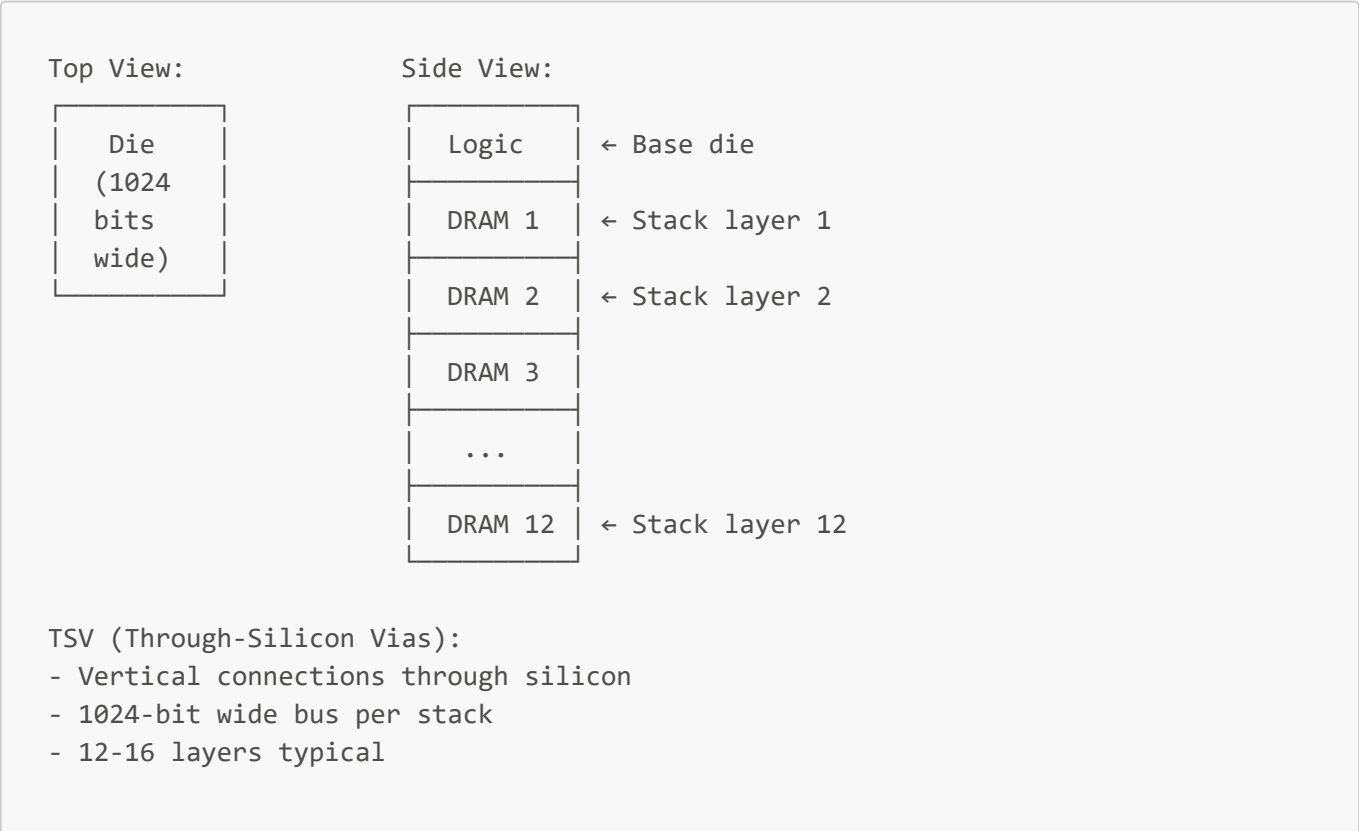
Side View:





HBM Stacking (3D)

HBM3 Structure:



- H100 HBM3:
- 5 stacks × 16GB = 80GB
 - 5 stacks × 3.35 TB/s = 3.35 TB/s total
 - Each stack: 671 GB/s

HBM vs GDDR:

Metric	HBM3	GDDR6X
Bandwidth	3.35 TB/s	1 TB/s
Bus Width	5120-bit	384-bit
Power	~50W	~50W
Cost	Very High	Medium
Die Space	Small	Large (PCB)
Use Case	AI/HPC	Gaming

- Why HBM for AI?
- Memory bandwidth is bottleneck
 - Worth the cost for training/inference

Die Cost Model

```
def calculate_die_cost(
    wafer_cost,
    die_size_mm2,
    defect_density=0.09,
    alpha=6,
    test_cost_per_die=50,
    packaging_cost=1000 # CoWoS is expensive!
):
    """
    Complete die cost model
    """
    # Dies per wafer
    dies_per_wafer = calculate_dies_per_wafer(die_size_mm2)

    # Yield
    die_yield = calculate_yield(die_size_mm2, defect_density, alpha)

    # Good dies per wafer
    good_dies = dies_per_wafer * die_yield

    # Wafer cost per good die
    wafer_cost_per_die = wafer_cost / good_dies

    # Total die cost
    total_cost = wafer_cost_per_die + test_cost_per_die + packaging_cost
```

```

    return {
        'wafer_cost_per_die': wafer_cost_per_die,
        'test_cost': test_cost_per_die,
        'packaging_cost': packaging_cost,
        'total_cost': total_cost,
        'dies_per_wafer': dies_per_wafer,
        'yield': die_yield,
        'good_dies': good_dies
    }

# H100 cost estimate
h100_cost = calculate_die_cost(
    wafer_cost=16000,      # TSMC 4N wafer
    die_size_mm2=814,
    defect_density=0.09,
    alpha=6,
    test_cost_per_die=100, # Complex testing
    packaging_cost=1500    # CoWoS + HBM3
)

print("H100 Die Cost Breakdown:")
print(f"  Dies per wafer: {h100_cost['dies_per_wafer']}")
print(f"  Yield: {h100_cost['yield']:.1%}")
print(f"  Good dies: {h100_cost['good_dies']:.1f}")
print(f"  Wafer cost per die: ${h100_cost['wafer_cost_per_die']:.0f}")
print(f"  Test cost: ${h100_cost['test_cost']:.0f}")
print(f"  Packaging cost: ${h100_cost['packaging_cost']:.0f}")
print(f"  Total die cost: ${h100_cost['total_cost']:.0f}")
print(f"\nNVIDIA sells H100 for: ~$30,000")
print(f"Gross margin: ~{(30000 - h100_cost['total_cost']) / 30000 * 100:.0f}%")

# Output:
# Dies per wafer: 80
# Yield: 71.9%
# Good dies: 57.5
# Wafer cost per die: $278
# Test cost: $100
# Packaging cost: $1500
# Total die cost: $1878
#
# NVIDIA sells H100 for: ~$30,000
# Gross margin: ~94%

```

Process Node Economics

Cost Comparison:

```

def compare_process_nodes():
    """Compare cost across process nodes"""

```

```

nodes = {
    '7nm': {'wafer_cost': 10000, 'defect_density': 0.08, 'density': 96},
    '5nm': {'wafer_cost': 16000, 'defect_density': 0.09, 'density': 171},
    '3nm': {'wafer_cost': 20000, 'defect_density': 0.12, 'density': 292}
}

# Fixed: 70B transistors, same as H100
target_transistors = 80e9

for node, specs in nodes.items():
    # Die size for 80B transistors
    density = specs['density'] * 1e6 # MTr/mm² to Tr/mm²
    die_size = target_transistors / density

    # Calculate cost
    cost = calculate_die_cost(
        wafer_cost=specs['wafer_cost'],
        die_size_mm2=die_size,
        defect_density=specs['defect_density'],
        packaging_cost=1500
    )

    print(f"\n{node}:")
    print(f"  Die size: {die_size:.0f} mm²")
    print(f"  Yield: {cost['yield']:.1%}")
    print(f"  Cost per die: ${cost['total_cost']:.0f}")
    print(f"  Transistor density: {specs['density']} MTr/mm²")

compare_process_nodes()

# Output shows:
# - 3nm has smallest die (best density)
# - But lower yield and higher wafer cost
# - Net result: Similar total cost!
# Trade-off: Performance vs Cost

```

5. Datacenter Infrastructure & TCO

Total Cost of Ownership (TCO)

Components:

TCO over 3 years:

1. Hardware CapEx
 - GPU/accelerator cost
 - Server cost (CPU, RAM, storage)
 - Networking (switches, cables)
 - Racks, PDUs

2. Facility CapEx
 - Datacenter construction
 - Power infrastructure
 - Cooling infrastructure
 - Security, fire suppression
3. Operating Expenses (OpEx)
 - Electricity (compute + cooling)
 - Maintenance
 - Staff salaries
 - Software licenses
4. Depreciation
 - Hardware lifecycle (3-5 years)
 - Facility depreciation (10-20 years)

TCO Calculation:

```
def calculate_tco(
    num_gpus,
    gpu_cost,
    server_cost_per_gpu=2000,
    networking_cost_per_gpu=500,
    facility_cost_per_gpu=1000,
    power_per_gpu_w=700,
    pue=1.3, # Power Usage Effectiveness
    electricity_cost_per_kwh=0.10,
    maintenance_rate=0.10, # 10% of CapEx per year
    years=3
):
    """
    Calculate 3-year TCO for GPU cluster
    """
    # CapEx
    hardware_capex = num_gpus * (gpu_cost + server_cost_per_gpu +
    networking_cost_per_gpu)
    facility_capex = num_gpus * facility_cost_per_gpu
    total_capex = hardware_capex + facility_capex

    # Power consumption
    compute_power_w = num_gpus * power_per_gpu_w
    total_power_w = compute_power_w * pue # PUE includes cooling
    total_power_kw = total_power_w / 1000

    # Electricity cost
    hours_per_year = 24 * 365
    annual_electricity = total_power_kw * hours_per_year *
    electricity_cost_per_kwh
    total_electricity = annual_electricity * years

    # Maintenance
    annual_maintenance = total_capex * maintenance_rate
```

```

total_maintenance = annual_maintenance * years

# Total TCO
total_tco = total_capex + total_electricity + total_maintenance

return {
    'num_gpus': num_gpus,
    'hardware_capex': hardware_capex,
    'facility_capex': facility_capex,
    'total_capex': total_capex,
    'annual_electricity': annual_electricity,
    'total_electricity': total_electricity,
    'total_maintenance': total_maintenance,
    'total_tco': total_tco,
    'tco_per_gpu': total_tco / num_gpus,
    'power_kw': total_power_kw
}

# Example: 1024 H100 cluster
h100_tco = calculate_tco(
    num_gpus=1024,
    gpu_cost=30000,
    server_cost_per_gpu=3000,
    networking_cost_per_gpu=1000,
    facility_cost_per_gpu=2000,
    power_per_gpu_w=700,
    pue=1.3,
    electricity_cost_per_kwh=0.10,
    years=3
)

print("1024 H100 Cluster TCO (3 years):")
print(f"  Hardware CapEx: ${h100_tco['hardware_capex']/1e6:.1f}M")
print(f"  Facility CapEx: ${h100_tco['facility_capex']/1e6:.1f}M")
print(f"  Total CapEx: ${h100_tco['total_capex']/1e6:.1f}M")
print(f"  Electricity (3yr): ${h100_tco['total_electricity']/1e6:.1f}M")
print(f"  Maintenance (3yr): ${h100_tco['total_maintenance']/1e6:.1f}M")
print(f"  Total TCO: ${h100_tco['total_tco']/1e6:.1f}M")
print(f"  TCO per GPU: ${h100_tco['tco_per_gpu']:, .0f}")
print(f"  Cluster power: {h100_tco['power_kw']/1000:.2f} MW")

# Output:
# Hardware CapEx: $34.8M
# Facility CapEx: $2.0M
# Total CapEx: $36.9M
# Electricity (3yr): $7.9M
# Maintenance (3yr): $11.1M
# Total TCO: $55.8M
# TCO per GPU: $54,512
# Cluster power: 0.93 MW

```

TCO Breakdown (Pie Chart - ASCII):

- Hardware is dominant cost
- But electricity adds 14% (non-trivial!)
- Energy efficiency matters for TCO

Calculation:

25 / 32

```

print("Cost per Token (H100 cluster):")
print(f"  Total tokens (3yr): {h100_cost_per_token['total_tokens']/1e12:.2f}
trillion")
print(f"  Cost per token: ${h100_cost_per_token['cost_per_token']:.8f}")
print(f"  Cost per 1M tokens:
${h100_cost_per_token['cost_per_million_tokens']:.2f}")

# Compare with OpenAI pricing:
print(f"\nOpenAI GPT-4 pricing:")
print(f"  Input: $5 per 1M tokens")
print(f"  Output: $15 per 1M tokens")
print(f"  Our cost: ${h100_cost_per_token['cost_per_million_tokens']:.2f}")
print(f"  Markup: {5 / h100_cost_per_token['cost_per_million_tokens']:.1f}x")

```

Power Usage Effectiveness (PUE)

Definition:

$$\text{PUE} = \text{Total Facility Power} / \text{IT Equipment Power}$$

Perfect efficiency: PUE = 1.0 (impossible)

Good datacenter: PUE = 1.2-1.3

Average datacenter: PUE = 1.5-1.8

Poor datacenter: PUE = 2.0+

Example:

IT equipment: 1 MW

Cooling: 200 kW

Lighting, UPS losses: 100 kW

Total: 1.3 MW

$$\text{PUE} = 1.3 \text{ MW} / 1 \text{ MW} = 1.3$$

PUE Impact:

```

def pue_impact(
    it_power_mw,
    electricity_cost_per_kwh,
    years=3
):
    """
    Show impact of PUE on costs
    """
    pue_values = [1.1, 1.3, 1.5, 1.8, 2.0]

    print("PUE Impact on Electricity Cost:")
    print(f"IT Power: {it_power_mw} MW")
    print(f"Electricity: ${electricity_cost_per_kwh}/kWh")

```

```
print()

for pue in pue_values:
    total_power = it_power_mw * pue
    annual_cost = total_power * 1000 * 24 * 365 * electricity_cost_per_kwh
    total_cost = annual_cost * years

    print(f"PUE {pue}:")
    print(f"  Total power: {total_power:.2f} MW")
    print(f"  3-year cost: ${total_cost/1e6:.2f}M")
    print()

pue_impact(it_power_mw=1.0, electricity_cost_per_kwh=0.10)

# Output shows:
# PUE 1.1: $2.89M
# PUE 1.3: $3.41M (+18%)
# PUE 1.5: $3.94M (+36%)
# PUE 2.0: $5.26M (+82%)
#
# Efficient cooling saves millions!
```

Stranded Power Problem

Challenge:

GPU power density increasing faster than facility power:

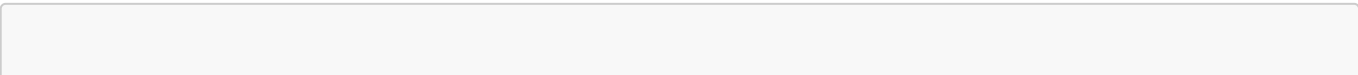
Rack Power Evolution				
Year	GPU		Rack Power	Problem
2018	V100 300W	8x =	2.4kW	✓ OK
2020	A100 400W	8x =	3.2kW	✓ OK
2022	H100 700W	8x =	5.6kW	⚠ Tight
2024	B100 1000W	8x =	8.0kW	✗ Exceeds!

Most datacenters designed for 5-10kW per rack.
New GPUs exceed this!

Solutions:

- 1. Liquid cooling (more power per rack)
- 2. Fewer GPUs per rack (wastes space)
- 3. Build new datacenters (expensive, slow)

Liquid Cooling:



Air cooling limit: ~30 kW per rack
Liquid cooling: 100+ kW per rack

Methods:

1. Direct-to-chip (cold plate on GPU)
2. Immersion cooling (submerge in liquid)
3. Rear-door heat exchangers

Cost:

- Retrofitting: \$50K-100K per rack
- New build: \$20K-40K per rack

But enables:

- 3-4× power density
- Better PUE (1.1-1.2)
- Utilize existing space

6. Memory Systems Deep Dive

HBM (High Bandwidth Memory) Architecture

HBM3 Detailed Specs:

HBM3 Stack (16GB)
Layers: 12 DRAM dies + 1 base logic die
Interface: 1024 bits per stack
Speed: 6.4 Gbps per pin
Bandwidth: $1024 \times 6.4 \text{ Gbps} / 8 = 819 \text{ GB/s}$
H100 configuration:
5 stacks $\times$ 819 GB/s = 4,095 GB/s
(NVIDIA spec: 3.35 TB/s with guardbands)
Power: ~8W per stack (~40W total)

HBM Evolution:

Gen	Speed	Per-Stack	Die Layers
HBM1	1.0 Gbps	128 GB/s	4
HBM2	2.0 Gbps	256 GB/s	8
HBM2e	3.6 Gbps	461 GB/s	8-12
HBM3	6.4 Gbps	819 GB/s	12
HBM3e	8.0 Gbps	1,024GB/s	12-16

HBM vs DRAM Comparison:

```
def compare_memory_types():
    """Compare HBM3 vs DDR5 vs GDDR6X"""

    memory_types = {
        'HBM3': {
            'bus_width': 5120, # 5 stacks × 1024-bit
            'speed_gbps': 6.4,
            'capacity_gb': 80,
            'power_w': 40,
            'cost_multiplier': 4
        },
        'DDR5': {
            'bus_width': 512, # 8 channels × 64-bit
            'speed_gbps': 4.8,
            'capacity_gb': 512,
            'power_w': 30,
            'cost_multiplier': 1
        },
        'GDDR6X': {
            'bus_width': 384,
            'speed_gbps': 21,
            'capacity_gb': 24,
            'power_w': 50,
            'cost_multiplier': 1.5
        }
    }

    print("Memory Comparison:\n")
    for mem_type, specs in memory_types.items():
        bandwidth = (specs['bus_width'] * specs['speed_gbps']) / 8 # GB/s
        bw_per_watt = bandwidth / specs['power_w']
        bw_per_dollar = bandwidth / specs['cost_multiplier']

        print(f"{mem_type}:")
        print(f"  Bandwidth: {bandwidth:.0f} GB/s")
        print(f"  Capacity: {specs['capacity_gb']} GB")
        print(f"  Power: {specs['power_w']} W")
        print(f"  BW/Watt: {bw_per_watt:.1f} GB/s/W")
        print(f"  BW/$ (relative): {bw_per_dollar:.0f}")
        print()

    compare_memory_types()

# Output shows HBM3 has:
# - 10× bandwidth vs DDR5
# - Better power efficiency
# - But 4× cost and limited capacity
```

Memory Hierarchy

Complete Memory Hierarchy:

Memory Hierarchy			
Level	Size	Latency	Bandwidth
Registers	64KB	0 cycles	Infinite
L1 Cache	256KB	1-2 cycles	~20 TB/s
Shared Mem	256KB	1-2 cycles	~20 TB/s
L2 Cache	50MB	~30 cycles	~7 TB/s
HBM	80GB	~300 cycles	3.35 TB/s
Host RAM	512GB+	~500 cycles	100 GB/s (PCIe)
SSD	10TB+	~100K cycles	14 GB/s

Analogy:
Registers = desk drawer (instant access)
L1/L2 = desk (fast)
HBM = bookshelf (slower but larger)
Host RAM = library (slow, huge)
SSD = warehouse (very slow, very large)

KV Cache Optimization Techniques

1. Multi-Query Attention (MQA)

Standard Multi-Head Attention:
Each head has separate K, V

Head 1: K_1, V_1
Head 2: K_2, V_2
...
Head 64: K_{64}, V_{64}

KV cache size: $\text{num_heads} \times \text{seq_len} \times \text{head_dim}$

Multi-Query Attention:
All heads share single K, V

Shared: K, V
Head 1: Q_1
Head 2: Q_2
...
Head 64: Q_{64}

KV cache size: $1 \times \text{seq_len} \times \text{d_model}$

Reduction: 64x smaller KV cache!

Trade-off:

- ✓ 64× less memory
- ✓ 64× less bandwidth
- ⚠ Slight quality degradation (~2-3%)

2. Grouped-Query Attention (GQA)

Middle ground: Groups of heads share K, V

LLaMA 2 70B uses GQA:

- 64 query heads
- 8 KV heads (8 groups)
- Each group: 8 query heads share 1 KV head

KV cache reduction: $64/8 = 8\times$ smaller

Better quality than MQA, still saves memory!

3. Quantization

```
def kv_cache_with_quantization(
    batch_size,
    seq_len,
    num_layers,
    num_kv_heads,
    head_dim,
    bits=8 # INT8 instead of FP16
):
    """
    Calculate KV cache with quantization
    """
    bytes_per_element = bits / 8

    # 2x for K and V
    size_bytes = (
        2 * batch_size * seq_len *
        num_layers * num_kv_heads * head_dim *
        bytes_per_element
    )

    return size_bytes

# LLaMA 2 70B with different quantization
configs = [
    ('FP16', 16),
    ('INT8', 8),
    ('INT4', 4),
]
```

```
for name, bits in configs:
    size = kv_cache_with_quantization(
        batch_size=32,
        seq_len=4096,
        num_layers=80,
        num_kv_heads=8, # GQA
        head_dim=128,
        bits=bits
    )
    print(f"{name}: {size/1e9:.2f} GB")
```

Output:

FP16: 20.97 GB

INT8: 10.49 GB (2× reduction)

INT4: 5.24 GB (4× reduction)